

Cuestionario de Estructuras y Utilidades de Python

Total de puntos 90/100

Este cuestionario está diseñado para poner a prueba tus conocimientos sobre los conceptos esenciales de la programación en Python.

El correo electrónico del destinatario (**leandrosebastiantroncoso@gmail.com**) se registró al enviar el formulario.

✓ ¿Cuál de los siguientes tipos de datos es inmutable en Python? *

5/5

- ☐ list
- ☒ tupla
- ☐ dict
- ☐ set



✓ ¿Cuál de las siguientes afirmaciones describe mejor una lista en Python? * 5/5

- ☐ Es una colección inmutable de elementos ordenados.
- ☒ Es una colección mutable y ordenada que permite elementos duplicados.
- ☐ Es una colección mutable de elementos no ordenados sin duplicados.
- ☐ Es una colección que asocia claves a valores.



✓ ¿Qué característica distingue a las tuplas de las listas? *

5/5

- ☐ Las tuplas son mutables y las listas son inmutables.
- ☒ Las tuplas son inmutables y las listas son mutables.
- ☐ Las tuplas utilizan corchetes y las listas paréntesis.
- ☐ No existe diferencia.



✓ Para acceder a un valor en un diccionario por su clave, ¿cuál de las siguientes sintaxis es la correcta?

*5/5

- ☐ mi_dict{clave}
- ☐ mi_dict[clave]
- ☐ mi_dict['clave']
- ☒ Tanto B como C (dependiendo de si la clave es una variable o una cadena literal)



✓ ¿Cuál de las siguientes expresiones utiliza correctamente una list comprehension para crear una lista con los cuadrados de los números del 1 al 10?

*5/5

- ☐ [x*x for x in range(1, 11)]
- ☐ list(x*x for x in range(1, 11))
- ☐ {x*x for x in range(1, 11)}
- ☒ Tanto A como B



✓ ¿Cuál de las siguientes opciones crea correctamente un set mediante comprensión? *5/5

- ☐ [x for x in range(1, 6)]
- ☒ {x for x in range(1, 6)}
- ☐ (x for x in range(1, 6))
- ☐ dict((x, x) for x in range(1, 6))



✓ ¿Cuál de las siguientes afirmaciones describe correctamente la mutabilidad de las listas en Python? *5/5

- ☐ Las listas son inmutables; cualquier operación que intente modificar un elemento resultará en un error de tipo TypeError.
- ☒ Las listas son mutables, lo que significa que se pueden modificar sus elementos individuales, añadir nuevos o eliminar existentes sin cambiar la identidad (id) del objeto original.
- ☐ Las listas son mutables, pero cada vez que se modifica un elemento, Python crea internamente una copia completa de la lista en una nueva ubicación de memoria.
- ☐ Las listas son inmutables, por lo que para cambiar un valor es obligatorio reasignar la variable a una nueva tupla.



✓ Si ejecutamos el código `list(range(2,10,2))`, ¿cuál será el resultado esperado? *5/5

- ☐ [2,4,6,8,10]
- ☒ [2,4,6,8]
- ☐ [4,6,8,10]
- ☐ [2,3,4,5,6,7,8,9]



✓ ¿Qué devuelve la función all(l) cuando se aplica sobre una lista l? *

5/5

- ☐ Devuelve True si al menos uno de los elementos de la lista es evaluado como verdadero.
- ☐ Devuelve una nueva lista conteniendo solo los elementos que son verdaderos.
- ☒ Devuelve True si todos los elementos de la lista son evaluados como verdaderos, o si la lista está vacía. ✓
- ☐ Devuelve el número total de elementos que son verdaderos dentro de la lista.

✓ Al realizar una 'Copia por Referencia' (l1=l2), ¿qué sucede si modificamos un elemento en l2? *5/5

- ☒ Se modifican tanto l1 como l2, porque ambas variables apuntan a la misma dirección de memoria. ✓
- ☐ Solo se modifica l2, ya que l1 mantiene una copia independiente de los datos originales.
- ☐ El programa arroja un error de compilación/ejecución por intentar duplicar punteros.
- ☐ Se crea automáticamente una nueva lista en l1 para preservar la integridad de los datos originales.



✓ ¿Cuál es la principal característica que distingue a las tuplas de las listas? *5/5

- ☐ Las tuplas pueden contener diferentes tipos de datos, mientras que las listas solo permiten un único tipo.
- ☐ Las listas utilizan paréntesis () para su definición, mientras que las tuplas utilizan corchetes [].
- ☐ Las tuplas ocupan mucho más espacio en memoria que las listas debido a su estructura compleja.
- ☒ Las tuplas son inmutables, lo que significa que no pueden modificarse una vez creadas, a diferencia de las listas. ✓

✓ Si utilizamos la función zip() con dos listas de diferentes longitudes, ¿cuándo se detiene el proceso de emparejamiento? *5/5

- ☐ El proceso continúa hasta completar la lista más larga, rellenando los huecos de la lista corta con valores None.
- ☐ Python lanza una excepción de tipo ValueError al detectar que las colecciones no tienen el mismo tamaño.
- ☒ El proceso se detiene en cuanto se agotan los elementos de la lista más corta, ignorando el resto de los elementos de la lista larga. ✓
- ☐ El proceso se detiene únicamente cuando ambas listas han sido recorridas por completo, duplicando el último elemento de la lista más corta.



✗ ¿Cuál de las siguientes propiedades es FALSA respecto a los Diccionarios en Python?

*0/5

- ☐ Los diccionarios son colecciones mutables, lo que permite agregar, eliminar o modificar elementos tras su creación.
- ☒ A partir de Python 3.7, los diccionarios mantienen el orden de inserción de los elementos como una característica oficial del lenguaje. ✗
- ☐ Los valores asociados a las claves pueden ser de cualquier tipo de datos y pueden estar duplicados dentro del mismo diccionario.
- ☐ Cualquier objeto de Python, incluyendo listas y otros diccionarios, puede ser utilizado como clave de un diccionario.

Respuesta correcta

- ☒ Cualquier objeto de Python, incluyendo listas y otros diccionarios, puede ser utilizado como clave de un diccionario.

✓ Dada la operación de conjuntos `mi_set_a.symmetric_difference(mi_set_b)`, ¿qué elementos resultarán en el nuevo set?

*5/5

- ☐ Todos los elementos que pertenecen a ambos conjuntos simultáneamente (la zona común).
- ☐ Todos los elementos de `mi_set_a` y `mi_set_b`, eliminando únicamente los duplicados exactos.
- ☒ Los elementos que están en `mi_set_a` o en `mi_set_b`, pero no en ambos al mismo tiempo. ✓
- ☐ Únicamente los elementos que se encuentran en `mi_set_a` pero que han sido descartados de `mi_set_b`.



✓ ¿Cuál es la función del parámetro 'inicio' en la función `enumerate(iterable, inicio)`?

*5/5

- ☐ Define el índice del elemento dentro del iterable donde la función debe empezar a iterar, saltándose los anteriores.
- ☒ Establece el valor numérico desde el cual comenzará el contador que acompaña a los elementos, sin afectar el orden del iterable. ✓
- ☐ Indica cuántas veces debe repetirse la iteración sobre el mismo objeto iterable.
- ☐ Determina el valor máximo que puede alcanzar el contador antes de detener la ejecución del bucle.

✓ En el ecosistema de Python, las **List Comprehensions** permiten crear listas de una manera muy compacta. A pesar de que un uso excesivo puede afectar la legibilidad o interpretabilidad del código, ¿qué ventaja principal justifica su uso frecuente?

*5/5

- ☒ Permiten que el código se ejecute más rápido que un bucle for tradicional debido a que están optimizadas a nivel interno en C. ✓
- ☐ Son la única forma en Python de crear listas que contengan más de un tipo de dato simultáneamente.
- ☐ Garantizan que la memoria del ordenador se libere automáticamente justo después de terminar la creación de la lista.
- ☐ Permiten ignorar las reglas de indentación de Python, facilitando la escritura de scripts en una sola línea.



✗ ¿Cuál es la diferencia fundamental entre las funciones `random.randrange(1, 10)` y `random.randint(1, 10)` al generar un número entero? *0/5

- ☐ `randint` incluye el número 10 en el rango de posibles resultados, mientras que `randrange` lo excluye.
- ☐ `randrange` incluye el 10, pero `randint` solo llega hasta el 9.
- ☐ No existe ninguna diferencia técnica; ambas generan un número aleatorio entre 1 y 10 inclusive.
- ☒ `randint` solo genera números enteros, mientras que `randrange` puede generar números flotantes (decimales). ✗

Respuesta correcta

- ☒ `randint` incluye el número 10 en el rango de posibles resultados, mientras que `randrange` lo excluye.

Comentarios

La función `randint(a, b)` es inclusiva en ambos extremos $[a,b]$, mientras que `randrange(a, b)` sigue la lógica de la función `range()` de Python, donde el límite superior es exclusivo $[a,b)$.

✓ Al trabajar con el módulo `datetime` de Python, ¿cuál es la diferencia principal entre los métodos `strftime()` y `strptime()`? *5/5

- ☒ `strftime` convierte un objeto de fecha en una cadena de texto (string), mientras que `strptime` crea un objeto de fecha a partir de una cadena de texto. ✓
- ☐ `strptime` se usa para obtener la hora actual del sistema, mientras que `strftime` se usa para cálculos matemáticos entre fechas.
- ☐ `strftime` solo funciona con fechas (años, meses, días) y `strptime` solo funciona con horarios (horas, minutos, segundos).
- ☐ No hay diferencia funcional; ambos métodos realizan la misma tarea y son intercambiables según la preferencia del programador.



✓ ¿Cuál de estas estructuras de datos es la única que NO mantiene un orden de inserción y además no permite elementos duplicados? *5/5

☐ Listas

☒ Sets

☐ Dicionarios

☐ Tuplas



✓ Si necesitas almacenar una colección de elementos que garantice que nadie podrá modificar, añadir o eliminar valores una vez creada, ¿qué estructura deberías elegir? *5/5

☐ Listas

☐ Sets

☐ Dicionarios

☒ Tuplas



El formulario se creó en Universidad de La Punta. - [Comunicarse con el propietario del formulario](#)

¿El formulario parece sospechoso? [Informe](#)

Google Formularios



